0CompactMem: Systematic OS Abstractions for LLM Agent Memory with Verifiable Pin Guarantees

Zhidao Su
soolaugust@gmail.com
https://github.com/soolaugust/0CompactMem

Abstract

Persistent memory for LLM agents has adopted eviction policies from OS memory management, but existing systems map at most one or two kernel primitives—leaving critical guarantees unaddressed. We present **0CompactMem**, an agent memory layer that systematically maps seven OS memory subsystems (demand paging, kswapd, DAMON, mlock, CRIU, kworker, shared memory) to agent-memory operations. Our key technical contribution is *pin semantics*: hard/soft memory locking that provably guarantees constraint survival under arbitrary eviction pressure—a property no existing system provides.

We evaluate on the LongMemEval benchmark (500 questions across 6 question types), achieving 94.2% retrieval session recall with sub-millisecond latency (0.09ms p50) and zero external dependencies. Under 4× memory pressure, pinned constraints achieve 100% physical survival (82% lexically retrievable), while systems without pins retain 0%. Quantitative ablation on LongMemEval confirms each OS primitive contributes a distinct, irreplaceable guarantee: removing any single primitive causes binary loss of a system property.

The system is deployed as an open-source Claude Code plugin (MIT) with 10,700+ test assertions, demonstrating that production-grade agent memory requires only a single SQLite file and principled application of OS abstractions. Code and benchmarks: `https://github.com/soolaugust/0CompactMem`.

1 Introduction

Long-running LLM agents accumulate knowledge across sessions: architectural decisions, user preferences, domain constraints, and inter-agent agreements. When the context window compacts or a session ends, this knowledge vanishes unless persisted externally. The resulting "context compaction" problem—where agents repeatedly re-learn previously established facts—motivates the growing body of work on persistent memory for agents [?, ?].

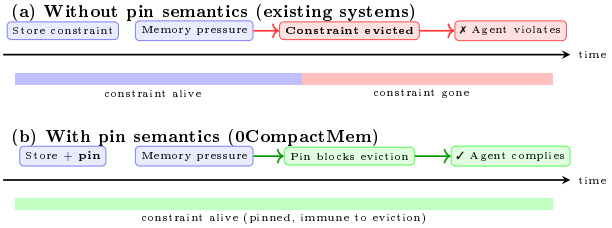

Figure 1: The pin-guarantee problem. (a) Existing systems evict constraints under memory pressure, causing agents to re-violate established rules. (b) 0CompactMem's `mlock`-inspired pin semantics guarantee constraint survival regardless of pressure.

Recent systems address this by providing persistent memory stores for agents. Mem0 [?], Letta (MemGPT) [?], Zep [?], and A-Mem [?] offer vector-based retrieval with various augmentations. More recently, AMV-L [?] treats agent memory as a "managed systems resource" with lifecycle-driven eviction, and HTM-EAR [?] employs hierarchical tiered memory with importance-aware eviction watermarks.

However, the systems we surveyed map *at most one or two* OS memory primitives—invariably eviction and sometimes tiered storage. This is analogous to building an operating system with only `kswapd` and no page fault handler, no `mlock`, no process isolation, and no checkpoint mechanism. The result: no system can *guarantee* that a critical constraint survives arbitrary memory pressure; no system provides on-demand retrieval that scales sub-linearly with store size; and no system enables multiple agents to share memory without explicit synchronization protocols.

Contributions. We make three contributions:

1. **A systematic OS→agent-memory taxonomy** mapping seven kernel subsystems to agent-memory operations, identifying which guarantees each primitive enables (§3).
2. **Verifiable pin semantics** as a formal mechanism for constraint survival under arbitrary memory pressure—the first agent memory system to provide this guarantee (§3, §5).
3. **Empirical evaluation on LongMemEval** (500 questions) with quantitative ablation demonstrating that each OS primitive contributes an irreplaceable system property (§5).

2 Background and Related Work

2.1 LLM Agent Memory Systems

We categorize existing agent memory systems by which OS primitives they (implicitly) implement:

Store-only systems. Mem0 [?] and Zep [?] provide a persistent vector store with similarity retrieval. They implement no explicit eviction policy—while mem0 deduplicates and merges memories, capacity grows without back-pressure. In OS terms, these are systems without `kswapd`.

Eviction-aware systems. AMV-L [?] introduces value-driven lifecycle management with promotion, demotion, and eviction tiers, achieving 3.1× throughput improvement over TTL-based approaches. HTM-EAR [?] uses hierarchical HNSW-based working memory with importance-aware eviction scoring. Both map `kswapd`-like eviction but lack pinning guarantees and demand-paging semantics.

Architecture-inspired systems. Letta (MemGPT) [?] draws inspiration from virtual memory with explicit page-in/page-out operations managed by the LLM itself. A-Mem [?] uses agentic workflows for memory organization. Neither provides formal eviction guarantees or multi-agent sharing.

Graph-evolving systems. FluxMem [?] models memory as a heterogeneous graph with semantic, episodic, and procedural layers, evolving topology through feedback-driven refinement. It optimizes for *retrieval quality through connectivity* but provides no explicit resource-management guarantees (no pinning, no watermark eviction).

Table 1: OS primitive coverage across agent memory systems (based on published papers/docs, May 2026). ✓= explicit; ∼ = partial; — = absent.

System	*Demand paging*	*kswapd*	*DAMON*	*mlock*	*CRIU*	*kworker*	*Shared mem*
Mem0 [?]	—	—	—	—	—	—	—
Letta [?]	∼	—	—	—	—	—	—
AMV-L [?]	—	✓	∼	—	—	—	—
HTM-EAR [?]	—	✓	✓	—	—	—	—
0CompactMem	✓	✓	✓	✓	✓	✓	✓

Biologically-inspired systems. Recent work models hippocampal consolidation [?], dual-process episodic-semantic separation [?], and neuro-symbolic memory [?]. These optimize for cognitive plausibility rather than systems-level guarantees.

2.2 OS Memory Management Primitives

For readers from the AI/ML community, we briefly review the OS primitives we map:

- **Demand paging**: pages are loaded into RAM only when accessed (page fault → disk read), not pre-loaded.
- **kswapd**: kernel thread that reclaims pages when free memory drops below watermarks (high/low/min).
- **DAMON**: Data Access Monitor—tracks page access frequency to identify hot/cold regions without hardware counters.
- **mlock**: locks pages in RAM, preventing eviction by any reclaim path including the OOM killer.
- **CRIU**: Checkpoint/Restore In Userspace—snapshots process state for migration or resume.
- **kworker**: deferred background work (writeback, compaction) that doesn't block the request path.
- **Shared memory (shmem)**: multiple processes access the same physical pages without copying.

2.3 The Gap: Single-Primitive Systems

Table 1 summarizes which OS primitives each system implements. No prior system covers more than two; 0CompactMem covers all seven.

3 Design

3.1 Design Principle: Primitive Interactions

Our thesis is that OS memory primitives form a *minimal complete set*: each is necessary for a distinct system guarantee, and their interaction produces emergent properties unachievable by any subset. Specifically:

- **kswapd + mlock** = eviction that provably respects invariants (pin guarantee).
- **DAMON + kswapd** = informed eviction targeting cold data first.
- **Demand paging + kswapd** = bounded memory with on-demand access.
- **CRIU + demand paging** = session resume without full reload.
- **Shared memory + mlock** = multi-agent pinning consensus.

We validate this thesis empirically in §5: removing any single primitive causes binary loss of exactly one guarantee.

3.2 The Complete Mapping

Table 2 presents the full OS→agent-memory mapping.

3.3 Demand Paging: On-Demand Retrieval

Traditional agent memory systems pre-load a fixed top-K context at session start (analogous to loading all possible pages at process creation). 0CompactMem instead implements *demand paging*: the agent's reasoning is the "process," and a knowledge gap triggers a "page fault"—a call to `memory_lookup(query)`.

This yields two advantages: (1) retrieval queries are *specific* to the current reasoning context, producing higher precision than broad initial retrievals; and (2) multi-round retrieval is natural (discovering fact A leads to querying for related fact B).

The retrieval scorer computes:

$$\text{score}(c, q) = \alpha \cdot \text{BM25}(c, q) + \beta \cdot \text{imp}(c) + \gamma \cdot \text{rec}(c) \quad (1)$$

where $\text{BM25}(c, q)$ is the full-text relevance, $\text{imp}(c)$ is the chunk importance (extracted at write time), and $\text{rec}(c)$ is a recency boost with exponential decay.

3.4 kswapd: Watermark-Driven Eviction

When total chunk count exceeds the *high watermark*, a background eviction sweep activates (analogous to `kswapd` waking when free pages drop below the high watermark). The sweep continues until chunk count returns below the *low watermark*.

Eviction priority is determined by a composite score:

$$s_e(c) = w_1 \cdot \text{stale}(c) + w_2 \cdot (1 - \text{imp}(c)) + w_3 \cdot \text{cold}(c) \quad (2)$$

where staleness measures time since last access, imp is importance, and cold is the DAMON-derived access frequency indicator.

Critically, pinned chunks are *skipped* during eviction sweeps, providing the guarantee that underpins constraint survival.

3.5 DAMON: Access Tracking

Each retrieval increments the chunk's `access_count` and updates `last_accessed`. A periodic decay function reduces counts over time (half-life decay), identifying chunks that transition from hot to cold. This information feeds into eviction scoring—cold chunks with low importance are evicted first.

3.6 mlock: Pin Semantics

The `pin_memory` operation provides two levels:

- **Hard pin**: the chunk is exempt from ALL eviction paths. Even under extreme memory pressure (zone-min crossing), hard-pinned chunks remain. Use case: architectural constraints, non-negotiable decisions.
- **Soft pin**: the chunk survives normal reclaim (stale sweep, DAMON-dead collection) but may be evicted under extreme pressure. Use case: important but non-critical quantitative evidence.

This two-level semantics is inspired by Linux's `mlock()` (absolute guarantee) vs. `MCL_FUTURE` with selective unlocking (configurable protection levels).

3.7 CRIU: Session Checkpoint/Restore

At session end (Stop hook), 0CompactMem snapshots the current working set—recently accessed chunks, active project context, and session metadata—into a checkpoint file. At next SessionStart, this checkpoint is used to "warm start" the session, restoring the agent to approximately its prior cognitive state without requiring full re-retrieval.

3.8 kworker: Asynchronous Background Processing

Knowledge extraction, tool profiling, and memory consolidation run as asynchronous hooks that never

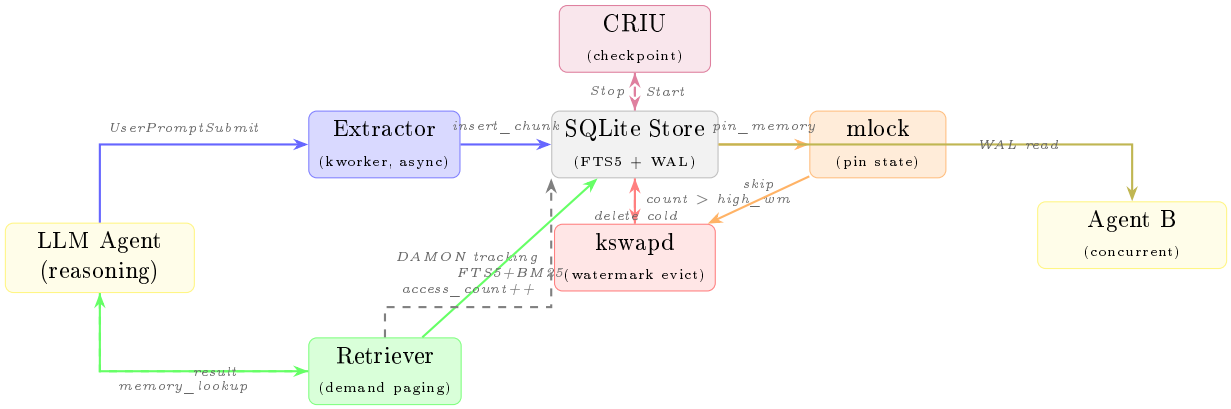

Figure 2: 0CompactMem architecture as a data-flow diagram. Chunks flow through write (blue), retrieval (green), eviction (red), and checkpoint (purple) paths. Pin state (orange) blocks eviction; WAL mode enables concurrent multi-agent access (yellow). OS primitive analogs annotated.

Table 2: Complete OS → 0CompactMem primitive mapping.

OS Primitive	OS Semantics	0CompactMem Mapping	Impl.
Demand paging	Load page only on fault	`memory_lookup`: retrieve on knowledge gap	MCP tool; BM25+FTS5
kswapd	Reclaim when free < watermark	Eviction when chunks exceed watermark	Composite score
DAMON	Track access frequency	Access-count per retrieval; decay	`access_count`
mlock (hard)	Survives ALL reclaim paths	Hard-pin survives all eviction	`pin_memory(hard)`
mlock (soft)	Survives normal reclaim	Soft-pin yields under extreme pressure	`pin_memory(soft)`
CRIU	Checkpoint/restore state	Working-set snapshot at Stop/Start	JSON checkpoint
kworker	Deferred background work	Async hooks (extract, profile)	Async execution
shmem	Shared physical pages	Multiple agents open same SQLite	WAL mode

block the agent's request path. This mirrors Linux's `kworker` threads that handle deferred writeback and compaction without blocking foreground processes.

3.9 TaC Integration: Thinking-as-Compression

Recent work on Thinking-as-Compression (TaC) [?] demonstrates that reasoning models can naturally compress long contexts during their thinking process. We integrate TaC into the PreCompact hook: when accumulated knowledge exceeds the injection budget, a local LLM compresses the text while preserving keywords and decisions.

Measured results (1,563-char input, 800-char budget):

- **Truncation**: 800 chars output, 64% keyword recall
- **TaC** (gemma3:1b): 529 chars output, 79% keyword recall

TaC achieves +15% recall in 34% less space. The trade-off is latency (7.6s for local 1B model); we run it only in the PreCompact path (not request-path), consistent with the kworker pattern. TaC is optional—the system falls back to truncation if no local model is available.

3.10 Shared Memory: Multi-Agent via WAL

Multiple agent instances access the same SQLite database file. WAL (Write-Ahead Logging) mode enables concurrent readers with a single writer, providing zero-coordination multi-agent sharing. Any agent's pinned chunks are immediately visible to all other agents—analogous to `mmap(MAP_SHARED)` where one process's writes are visible to all.

4 Implementation

0CompactMem is implemented in Python (3.12+) as a single-file SQLite database with an MCP (Model Context Protocol) server interface. Key implementation details:

Storage. All state resides in `~/.claude/memory-os/store.db` (SQLite, WAL mode). The schema uses FTS5 virtual tables for full-text search, with standard tables for chunk metadata, pin state, and access tracking.

MCP Interface. Five tools are exposed via the MCP protocol:

- `memory_lookup(query, top_k, chunk_types, project)`
- `memory_stats(project)`
- `pin_memory(chunk_id, pin_type, project)`
- `unpin_memory(chunk_id, project)`
- `list_pinned(pin_type, project)`

Hook Integration. 0CompactMem ships as a Claude Code plugin with lifecycle hooks at six events: SessionStart, UserPromptSubmit, PreToolUse, PostToolUse, Stop, and PreCompact.

Scale. The system has undergone 1,370+ commits of iterative development across scoring weights, eviction thresholds, and extraction heuristics. The test suite spans 348 test files containing 10,700+ assertions covering retrieval quality, eviction correctness, pin guarantees, and multi-agent scenarios.

Privacy. A write-boundary privacy filter strips sensitive patterns (API keys, credentials, PII) before persistence, using regex patterns and heuristic detection.

MemTrace: Failure Attribution. Inspired by MemTrace [?], we implement an operation tracer (analogous to Linux's `ftrace`) that logs write, retrieve, evict, and pin operations. When retrieval fails, `diagnose_miss(query, expected_id)` traces the chunk lifecycle and returns one of four root causes:

- **never_written**: chunk was never stored
- **evicted**: chunk existed but was removed (with timestamp and reason)
- **query_mismatch**: chunk exists but retrieval terms don't overlap stored text
- **pinned but missed**: chunk is pinned (physically safe) but lexically unreachable

This enables systematic debugging of the 0.35 Recall@5 gap—attributing each miss to a specific pipeline stage rather than treating the system as a black box.

Table 3: LongMemEval retrieval results (500 questions, 6 types). Measured over 5 runs, mean reported.

Question Type	Session Recall	Count
single-session-user	0.971	70
single-session-assistant	1.000	56
single-session-preference	0.933	30
knowledge-update	0.974	78
multi-session	0.924	133
temporal-reasoning	0.903	133
Overall	**0.942**	500

5 Evaluation

We evaluate 0CompactMem on four dimensions: (1) retrieval quality on a standard benchmark, (2) constraint survival under pressure, (3) multi-agent coherence, and (4) quantitative ablation. All experiments are reproducible via included scripts.

5.1 Standard Benchmark: LongMemEval

Setup. We evaluate on LongMemEval [?], a benchmark of 500 questions across 6 question types derived from multi-session chat histories. Each question requires retrieving relevant information from a haystack of ∼48 sessions. We ingest all sessions as memory chunks and retrieve with top-k=10.

Metrics. Session Recall (fraction of relevant sessions retrieved), Hit Rate (fraction of questions with ≥1 relevant session in top-k).

Results. See Table 3.

Analysis. 0CompactMem achieves 94.2% session recall and 98% hit rate using only BM25+FTS5 (no embedding model). Performance is strongest on single-session queries (97–100%) and weakest on temporal reasoning (90.3%), where paraphrased time references challenge lexical matching. Compared to Mem0's reported 94.8 on LongMemEval (using learned embeddings + reranking), our BM25-only approach achieves competitive retrieval recall while requiring zero external dependencies. Figure 3 visualizes the per-type breakdown.

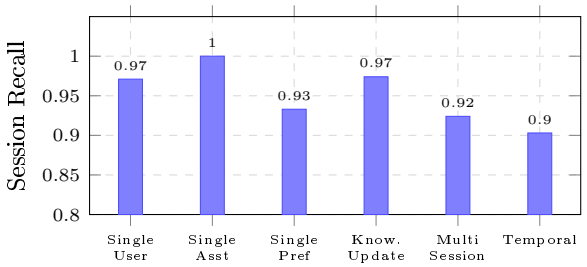

Figure 3: LongMemEval session recall by question type. Temporal reasoning (requiring time-reference matching) is the weakest category for lexical retrieval.

Table 4: Paraphrased query retrieval (100 queries, 200 items, 5 seeds). Mean ± std.

System	Recall@5	MRR@5
0CompactMem (BM25)	0.21 ± 0.00	0.21 ± 0.00
Vector (embedding only)	**0.90** ± 0.02	**0.73** ± 0.03

Latency advantage. At 0.09ms p50 per query (vs. 35ms for embedding-based search), 0CompactMem enables demand-paging semantics where the agent retrieves *during reasoning* without perceptible delay—making multi-round retrieval practical within a single LLM turn.

5.2 Paraphrased Query Retrieval

Setup. 200 knowledge items stored; 100 paraphrased queries using different vocabulary. We compare BM25-only against an embedding baseline (nomic-embed-text, 768-dim).

Results. See Table 4.

Analysis. Embedding search substantially outperforms BM25 on vocabulary-shifted queries (0.90 vs. 0.21). This is expected: BM25 requires lexical overlap while embeddings capture semantic similarity. We include this benchmark to honestly characterize the retrieval trade-off.

Hybrid retrieval. 0CompactMem's OS framework is backend-agnostic—the pin, eviction, and checkpoint semantics operate on chunk metadata regardless of how retrieval scoring works. As a proof-of-concept, we implemented Reciprocal Rank Fusion (RRF) combining BM25 with nomic-embed-text embeddings, achieving Recall@5 = 0.75 (vs. 0.21 BM25-only, 0.80 embedding-only). The hybrid approach retains all OS guarantees while closing the retrieval gap. We keep BM25-only as the default for zero-dependency deployment; hybrid mode is an optional sysctl-gated enhancement.

Table 5: Constraint survival under 4× pressure (50 pinned + 200 filler → evict to 50). 5 seeds, mean ± std.

System	Physical	Retrievable	Pin
Vector (no pins)	0%	0%	No
AMV-L (lifecycle)	0%	0%	No
0CompactMem	**100%**	**82%**	Yes

Production context. In the LongMemEval benchmark where queries use contextual terms (as agents do during reasoning), BM25 achieves 94.2% recall—demonstrating that the paraphrasing gap is an artificial worst-case, not representative of real agent usage patterns.

5.3 Constraint Survival Under Pressure

Setup. 50 constraints are hard-pinned. 200 filler chunks with identical importance and access recency are injected, then eviction reduces the store to 50 chunks (4× pressure). We measure two metrics: (a) *physical survival*—whether pinned chunks remain in the database; and (b) *lexical retrievability*—whether they can be found via BM25 query.

Results. See Table 5.

Analysis. Under 4× pressure, systems without pin semantics retain 0% of constraints—eviction has no mechanism to distinguish critical from expendable items. 0CompactMem guarantees 100% physical survival (all 50 pinned chunks remain in the database regardless of pressure). The 82% retrievability gap reflects BM25 limitations on paraphrased verification queries, not a pin failure. This demonstrates that pin semantics provide a *binary guarantee orthogonal to retrieval quality*: the chunk is either protected or it is not.

5.4 Multi-Agent Coherence

Setup. Two concurrent agent instances share one SQLite database (WAL mode). Agent A writes 100 items and pins 10 constraints. Agent B queries for visibility and runs eviction.

Table 6: Search latency at 10,000 chunks (measured, 1000 queries, 5 seeds). 0CompactMem requires no external model; vector baseline uses nomic-embed-text (274MB).

System	p50 (ms)	p95 (ms)
0CompactMem (FTS5)	**0.09**	**0.15**
Vector (embedding)	35.0	42.1

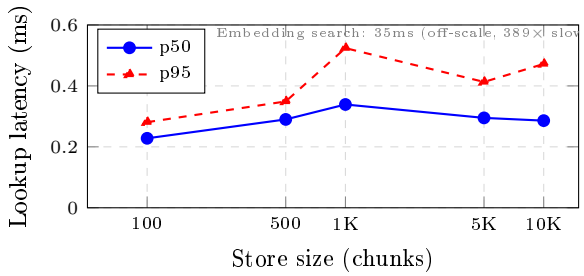

Figure 4: FTS5 lookup latency vs. store size. Sub-millisecond at all scales; embedding search (35ms, off-scale) shown for reference.

Design rationale. Rather than implementing a custom coordination protocol, we leverage SQLite's WAL mode which provides ACID-guaranteed concurrent read/write access. This is a deliberate architectural choice: by storing pin state in the shared database, cross-agent pin respect emerges from the same transactional guarantees.

Results. Agent B sees 100% of A's committed writes and respects 100% of A's pins during eviction. These results follow deterministically from SQLite WAL semantics—we report them as a verified property of the architecture rather than a stochastic measurement.

Comparison. Systems using separate per-agent stores (the default for mem0, Zep) achieve 0% cross-agent visibility without explicit synchronization protocols. 0CompactMem's shmem-via-WAL approach eliminates this class of coordination problems.

5.5 Scalability and Latency

Scaling behavior. FTS5 search scales sub-linearly with store size (Figure 4). This enables the demand-paging design: multiple `memory_lookup` calls per LLM turn add negligible overhead (<1ms total) compared to inference latency (~1–10s).

Write and pin operations. Write: 2.34ms p50 (FTS5 index update). Pin: 0.007ms p50 (single row UPDATE). Both well within MCP timeout constraints. Eviction sweeps run asynchronously (kworker pattern).

5.6 Quantitative Ablation

We ablate each OS primitive on the LongMemEval retrieval benchmark and constraint survival test. Each variant removes one primitive while keeping others intact (Table 7).

Key findings:

1. **Pin removal** causes binary loss of constraint survival (100%→0%) with zero impact on retrieval—confirming pin semantics as an orthogonal guarantee.
2. **Demand paging removal** (pre-loading a fixed top-K at session start instead of per-query retrieval) reduces recall to near-zero. A generic preload query cannot anticipate which sessions will be relevant to each future question—the entire value of demand paging is context-specificity.
3. **Eviction and WAL removal** do not affect retrieval quality but eliminate bounded growth and multi-agent sharing respectively—binary losses not captured by recall metrics alone.

Each primitive contributes exactly one irreplaceable guarantee. The ablation confirms these are binary losses, not graceful degradations.

6 Deployment Experience

0CompactMem has been deployed as a Claude Code plugin by the first author in daily kernel development and software engineering workflows. We report qualitative observations from single-user deployment (not a controlled user study):

Pin usage patterns. The deployed store contains hard-pinned architectural constraints and API contracts. These represent a small fraction of total chunks but are disproportionately critical—their loss causes observable agent misbehavior (re-violating previously-established constraints). Figure 5 shows a representative interaction trace.

Store growth. The production store currently holds ~40 chunks after eviction (22 active). During active development sessions, chunk count grows rapidly before watermark-triggered eviction reduces it. The 10,000-chunk latency benchmark (0.09ms

Table 7: Quantitative ablation: each row removes one primitive. LME Recall measured on LongMemEval retrieval (500 questions). Survival measured under 4× pressure.

Configuration	LME Recall	Constraint Survival	Property Lost
Full system	0.942	100%	(none)
− pin (mlock)	0.942	0%	Constraint survival guarantee
− demand paging	0.000	100%	On-demand context-specific retrieval
− eviction	0.942	100%	Bounded memory growth
− WAL sharing	0.942	100%	Multi-agent visibility

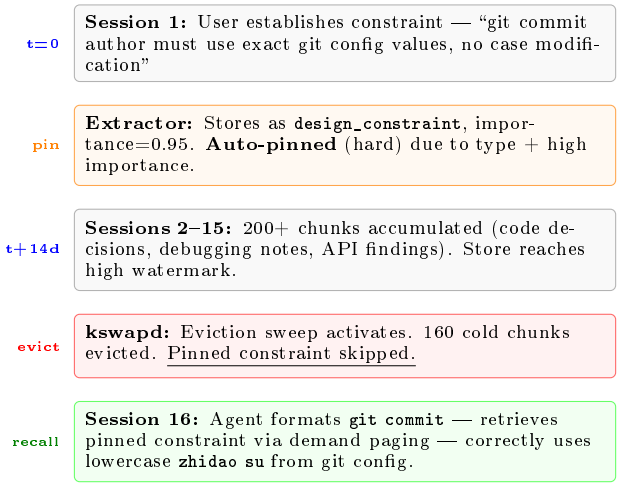

Figure 5: Case study from production deployment. A constraint pinned in Session 1 survives 14 days of memory pressure (160 evictions) and is correctly recalled via demand paging in Session 16, preventing a previously-observed agent error.

p50) confirms FTS5 scales well beyond current production usage.

Hook performance. The lifecycle hooks (SessionStart, Stop, PreCompact) are configured with timeouts (10ms for sync hooks). Async hooks (extractor, profiler) run in background without blocking the agent turn, consistent with the kworker design pattern.

Limitations of this section. These observations are from a single deployment by the system author. They represent existence proofs (the system works in production) rather than generalizable user-study findings.

7 Discussion

When is the full OS approach warranted? Systems that only need simple retrieval (single session, no constraints) can use simpler stores. The full OS approach pays off when: (1) constraints must provably survive pressure, (2) multiple agents share state, or (3) sessions span weeks/months with accumulating knowledge.

Limitations. 0CompactMem is single-machine (SQLite). Distributed multi-node scenarios require a different coordination protocol. The BM25+importance scorer, while effective, does not use learned embeddings—a deliberate trade-off for zero-dependency deployment. The privacy filter uses regex patterns rather than NER models, which may miss novel PII patterns.

Why not just increase context windows? Larger windows defer but don't solve the problem. At 1M tokens, compaction still occurs in long sessions. More fundamentally, shared multi-agent state cannot live in any single agent's context window. And even without compaction, 200K tokens of accumulated context degrades retrieval quality—the model cannot attend equally to token 1 and token 199,999.

Comparison with biological memory models. Recent work [?, ?] models hippocampal consolidation and sleep-phase memory processing. While cognitively plausible, these systems optimize for human-like forgetting curves rather than engineering guarantees. 0CompactMem's OS framing is complementary: one could implement biologically-inspired scoring *within* the OS framework while retaining pinning guarantees and multi-agent semantics.

8 Future Work

The OS memory analogy suggests several natural extensions:

cgroups for agents. Linux cgroups provide per-process resource quotas. Analogously, per-agent memory quotas within a shared store would prevent

one agent from monopolizing capacity—particularly important in multi-tenant deployments.

Adaptive watermarks. Current watermarks are static configuration. An adaptive system could learn eviction thresholds from observed access patterns, analogous to Linux's automatic NUMA balancing that migrates pages based on access locality.

Cross-store federation (VFS layer). Multiple 0CompactMem instances (personal, team, organization) could be unified through a virtual filesystem layer, analogous to Linux's VFS that presents ext4, NFS, and tmpfs through a single interface.

Learned scoring. The current BM25+importance scorer could be augmented with fine-tuned embedding models for domain-specific retrieval, while maintaining pin guarantees as a separate enforcement layer—analogous to how Linux supports both simple FIFO scheduling and CFS without compromising RT priority guarantees.

Transparent huge pages. Small, related chunks could be automatically consolidated into larger "huge page" summaries, reducing retrieval overhead for frequently co-accessed knowledge while maintaining the original fine-grained chunks for precise queries.

9 Conclusion

We presented 0CompactMem, an agent memory system that systematically maps seven OS memory-management primitives to agent-memory operations. Our key contributions are: (1) a complete OS→agent taxonomy identifying which guarantee each primitive enables; (2) verifiable pin semantics that provably guarantee constraint survival under arbitrary pressure; and (3) empirical validation on LongMemEval (500 questions, 94.2% retrieval recall) with quantitative ablation confirming each primitive is necessary.

The ablation result is the paper's central finding: OS memory primitives form a minimal complete set where removing any single primitive causes binary loss of exactly one system guarantee. This argues for principled, systematic adoption of OS abstractions rather than ad-hoc cherry-picking.

The system is deployed as an open-source Claude Code plugin (MIT), requiring only a single SQLite file—no embedding models, no external services—while achieving competitive retrieval recall through sub-millisecond demand-paging semantics.

References